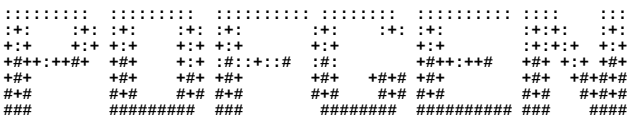
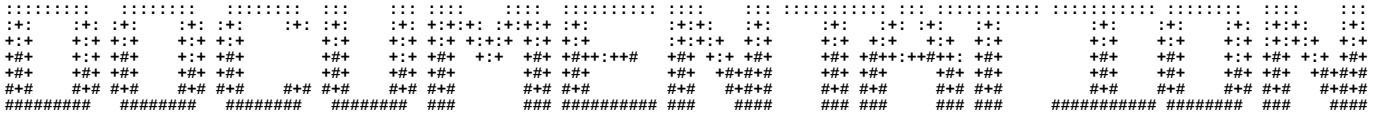




Purpose:

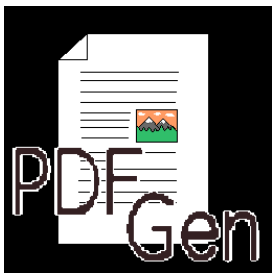
This library is designed to allow a user to procedurally generate PDF files.
It is a low level so the user has maximum control over the appearance of the document.

Features:

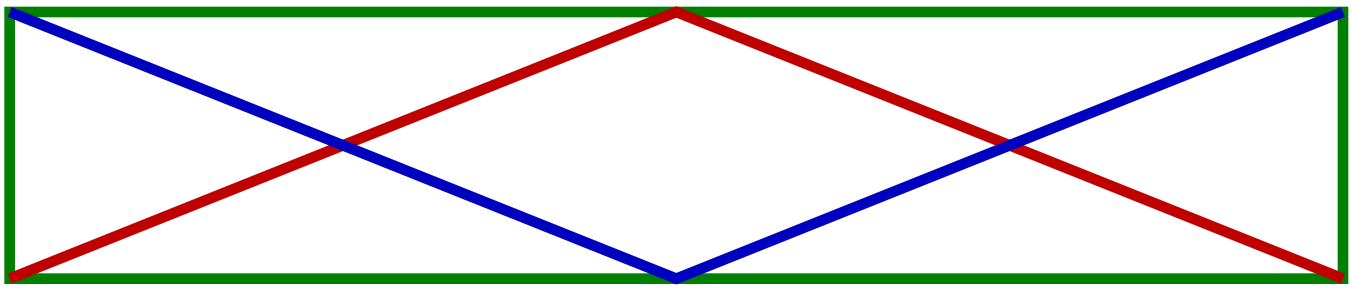
Currently it only supports the 14 type 1 fonts that are built-in to most viewers.

* Times-Roman
* Times-Bold
* Times-Italic
* Times-BoldItalic
* Helvetica
* Helvetica-Bold
* Helvetica-Oblique
* Helvetica-BoldOblique
* Courier
* Courier-Bold
* Courier-Oblique
* Courier-BoldOblique
* Symbol
* ZapfDingbats

Images can be imported scaled and rotated in the document. Most image types are supported.



Multicolor vector graphics can also be implemented.



Library Subs

SUB pdfAddPage

Adds a page to the document. A page must always be added. Once a new page is added you cannot edit previous pages.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF AddPage Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage

pdfBeginText
pdfSetFontSize "F1", 64
pdfSetPosition 120, 350
pdfAddText "This is a page"
pdfEndText

pdfAddPage

pdfBeginText
pdfSetFontSize "F1", 64
pdfSetPosition 35, 350
pdfAddText "This is another page"
pdfEndText

pdfGen "pdfAddPageExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfLoadFont (nm AS STRING, fnt AS STRING)

Loads an internal font and assigns an ID. Fonts must be loaded before use. Constants have been included in the Library.

* Times-Roman
* Times-Bold
* Times-Italic
* Times-BoldItalic
* Helvetica
* Helvetica-Bold
* Helvetica-Oblique
* Helvetica-BoldOblique
* Symbol
* ZapfDingbats
* Courier
* Courier-Bold
* Courier-Oblique
* Courier-BoldOblique

Parameters

nm - Id that the user assigns to the font ex. F1
fnt - name of internal font ex. Times-Roman

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
 pdfSetTitle "PDF Load Font Example"
 pdfSetAuthor "Justsomeguy"
 pdfLoadFont "F1", PDF_FONT_COURIER
 pdfLoadFont "F2", PDF_FONT_COURIER_BOLD
 pdfLoadFont "F3", PDF_FONT_COURIER_OBLIQUE
 pdfLoadFont "F4", PDF_FONT_COURIER_BOLD_OBLIQUE
 pdfLoadFont "F5", PDF_FONT_HELVETICA
 pdfLoadFont "F6", PDF_FONT_HELVETICA_BOLD
 pdfLoadFont "F7", PDF_FONT_HELVETICA_OBLIQUE
 pdfLoadFont "F8", PDF_FONT_HELVETICA_BOLD_OBLIQUE
 pdfLoadFont "F9", PDF_FONT_SYMBOL
 pdfLoadFont "F10", PDF_FONT_TIMES_ROMAN
 pdfLoadFont "F11", PDF_FONT_TIMES_BOLD
 pdfLoadFont "F12", PDF_FONT_TIMES_ITALIC
 pdfLoadFont "F13", PDF_FONT_TIMES_BOLD_ITALIC
 pdfLoadFont "F14", PDF_FONT_ZAPFDINGBATS
 pdfAddPage
 pdfBeginText
 pdfSetFontSize "F1", 12
 pdfSetLeading 15
 pdfSetPosition 40, 700
 pdfAddText "Courier": pdfNextLine
 pdfSetFontSize "F2", 12
 pdfAddText "Courier Bold": pdfNextLine
 pdfSetFontSize "F3", 12
 pdfAddText "Courier Oblique": pdfNextLine
 pdfSetFontSize "F4", 12
 pdfAddText "Courier Bold Oblique": pdfNextLine
 pdfSetFontSize "F5", 12
 pdfAddText "Helvetica": pdfNextLine
 pdfSetFontSize "F6", 12
 pdfAddText "Helvetica Bold": pdfNextLine
 pdfSetFontSize "F7", 12
 pdfAddText "Helvetica Oblique": pdfNextLine
 pdfSetFontSize "F8", 12
 pdfAddText "Helvetica Bold Oblique": pdfNextLine
 pdfSetFontSize "F1", 12
 pdfAddText "Symbol  ": pdfSetFontSize "F9", 12: pdfAddText "abcdefgh":
pdfNextLine
 pdfSetFontSize "F10", 12
 pdfAddText "Times Roman": pdfNextLine
 pdfSetFontSize "F11", 12
 pdfAddText "Times Bold": pdfNextLine
 pdfSetFontSize "F12", 12
 pdfAddText "Times Italic": pdfNextLine
 pdfSetFontSize "F13", 12
 pdfAddText "Times Bold Italic": pdfNextLine
 pdfSetFontSize "F1", 12
 pdfAddText "ZAPFDINGBATS  ": pdfSetFontSize "F14", 12: pdfAddText "abcdefgh":
pdfNextLine
 pdfEndText
 pdfGen "pdfLoadFontExample.pdf"
 SYSTEM
 '$INCLUDE:'pdfGen.bm'
```

SUB pdfSetPosition (x AS LONG, y AS LONG)

Sets the position where text will be drawn. Positions are relative.

Parameters

x - position
y - position

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF SetPosition Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage

pdfBeginText
pdfSetFontSize "F1", 24
pdfSetPosition 120, 350
pdfAddText "This is 120,350."
pdfSetPosition 60, 60
pdfAddText "This is 60, 60 from previous position."
pdfEndText

pdfGen "pdfSetPositionExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfAddText (s AS STRING)

Adds text to the page. Good for short text.

Parameters

s - Text string

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF AddText Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 24
pdfSetPosition 200, 350

pdfAddText "This is some short text."

pdfEndText

pdfGen "pdfAddTextExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfAddTextBlock (l AS LONG, s AS STRING)

Adds text to the page. Will start a new line after exceeds a specified length.

Parameters

l - Length in characters to start a new line.
s - Text string

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF AddTextBlock Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 24
pdfSetPosition 30, 450
pdfSetLeading 26 'Must be set before calling pdfAddTextBlock

'Moby Dick - Herman Melville
pdfAddTextBlock 60, "Call me Ishmael. Some years ago - never mind how long precisely - having little or no money in my purse, and nothing particular to interest me on shore, I thought I would sail about a little and see the watery part of the world."

pdfEndText

pdfGen "pdfAddTextBlockExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

FUNCTION pdfAddTextBlockEx$ (lmax AS LONG, lcMax AS LONG, s AS STRING)

Adds text to the page. Will start a new line after exceeds a specified length. If the line count exceeds lcMax the function will return the left over text.

Parameters

lmax - Length in characters to start a new line.
lcMax - Number of lines before returning the left over text.
s - Text string

Returns

Returns a string with the left over text.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF AddTextBlockEx Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfBeginText
pdfSetPosition 30, 720
pdfSetFontSize "F1", 18
pdfSetLeading 20
FOR l& = 1 TO 400: bigString$ = bigString$ + RandomWord + " ": NEXT
lineCount& = 33 ' Set the line count max.
charCount& = 48
DO
  bigString$ = pdfAddTextBlockEx(charCount&, lineCount&, bigString$)
  IF LEN(bigString$) > 0 THEN ' Start a new page
    pdfEndText 'End the text block
    pdfAddPage ' Add a page
    pdfBeginText ' Begin New Text Block
    pdfSetFontSize "F1", 18
    pdfSetPosition 30, 720
    pdfSetLeading 20
  END IF
LOOP WHILE LEN(bigString$) > 0
pdfEndText
pdfGen "pdfAddTextBlockEx_Example.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'

FUNCTION RandomWord$
  consonants$ = "BCDFGHJKLMNPRSTVWYZ": vowels$ = "AEIOU"
  Z& = (RND * 2) + 2
  FOR X& = 1 TO Z&
    B& = (RND * 19) + 1: c$ = c$ + MID$(consonants$, B&, 1)
    IF LEN(c$) > 4 THEN vowels$ = "AEIOUY"
    B& = (RND * 4) + 1: c$ = c$ + MID$(vowels$, B&, 1)
  NEXT
  B& = (RND * 4)
  IF B& >= 1 AND LEN(c$) > 5 THEN c$ = LEFT$(c$, LEN(c$) - 1)
  RandomWord$ = c$
END FUNCTION
```

SUB pdfBeginText

Starts a text block. All text operations must be done within a block. Resets position back to lower left corner.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF BeginText Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage

pdfBeginText

pdfSetFontSize "F1", 14
pdfSetPosition 150, 350 'Initial position
pdfAddText "This is some text."
pdfSetPosition 20, 20 'Relative position
pdfAddText " This some more text."
pdfEndText

pdfBeginText 'Resets back to lower left

pdfSetFontSize "F1", 14
pdfSetPosition 150, 300
pdfAddText "This is some text."
pdfSetPosition 20, 20 'Relative position
pdfAddText " This some more text."
pdfEndText

pdfGen "pdfBeginTextExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfEndText

Closes a text block.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF EndText Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage

pdfBeginText 'Begin Text Block
pdfSetFontSize "F1", 14
pdfSetPosition 150, 350 'Initial position
pdfAddText "This is some text."

pdfEndText 'End Text Block

pdfGen "pdfEndTextExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfNextLine

Acts as a carriage return after a text has been added. The distance moved down is set by pdfSetLeading xx.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF NextLine Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition 150, 350 'Initial position
pdfSetLeading 15

pdfAddText "This is some text."
pdfAddText " This is some more text."
pdfNextLine
pdfAddText "This is some text on a new line."
pdfEndText

pdfGen "pdfNextLineExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetRenderMode (m AS LONG)

The text rendering mode determines whether showing text causes glyph outlines to be stroked, filled, used as a clipping boundary, or some combination of the three.

PDF_RENDER_MODE_FILL - Fill text.
PDF_RENDER_MODE_STROKE - Stroke text.
PDF_RENDER_MODE_FILL_STROKE - Fill, then stroke text.
PDF_RENDER_MODE_INVISIBLE - Neither fill nor stroke text (invisible).
PDF_RENDER_MODE_FILL_ADD_PATH - Fill text and add to path for clipping
PDF_RENDER_MODE_STROKE_ADD_PATH - Stroke text and add to path for clipping.
PDF_RENDER_MODE_FILL_STROKE_ADD_PATH - Fill, then stroke text and add to path for clipping.
PDF_RENDER_MODE_ADD_PATH - Add text to path for clipping.

Parameters

m - mode

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF RenderMode Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_BOLD
pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 64
pdfSetPosition 200, 414 'Initial position
pdfSetStrokeWidth .25
pdfSetRenderMode PDF_RENDER_MODE_STROKE_ADD_PATH
pdfAddText "pdfGEN"
pdfEndText

pdfBeginText
pdfSetPosition 200, 480 'Initial position
pdfSetFontSize "F1", 6: pdfSetLeading 6
pdfSetRenderMode PDF_RENDER_MODE_FILL
FOR i& = 0 TO 1000: s$ = s$ + "clipped": NEXT
pdfAddTextBlock 100, s$
pdfEndText

pdfGen "pdfRenderModeExample.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetGrayScaleStroke (g AS SINGLE)

Sets the color space to grayscale and stroke to grayscale color.

Parameters

Grayscale color 0.0-1.0

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF GrayScaleStroke Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
c# = 0.0
FOR y& = 1 TO 5
  pdfSetStrokeWidth 10
  pdfSetGrayScaleStroke c#
  pdfBeginPath 100, 550 - y& * 20
  pdfAppendLine 500, 550 - y& * 20
  pdfStrokePath

  pdfSetGrayScaleStroke 0
  pdfSetStrokeWidth 1
  pdfRectangle 100, 545 - y& * 20, 400, 10
  pdfStrokePath

  pdfBeginText
  pdfSetFontSize "F1", 10
  pdfSetPosition 240, 547 - y& * 20
  pdfSetGrayScaleNonStroke (1 - c#) * -(y& <> 3)
  pdfAddText "Grayscale : " + STR$(c#)
  pdfEndText
  c# = c# + .25
NEXT
pdfGen "pdfGrayScaleStroketExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetGrayScaleNonStroke (g AS SINGLE)

Sets the color space to grayscale and non-stroke to grayscale color.

Parameters

Grayscale color 0.0-1.0

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF GrayScaleNonStroke Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
c# = 0.0
FOR y& = 1 TO 5
pdfSetStrokeWidth 10
pdfSetGrayScaleStroke c#
pdfBeginPath 45, 650 - y& * 20
pdfAppendLine 120, 650 - y& * 20
pdfStrokePath

pdfBeginText
pdfSetFontSize "F1", 10
pdfSetPosition 50, 647 - y& * 20
pdfSetGrayScaleNonStroke (1 - c#) * -(y& <> 3)
pdfAddText "Grayscale : " + STR$(c#)
pdfEndText
c# = c# + .25
NEXT
pdfGen "pdfGrayScaleNonStrokeExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetStrokeWidth (sw AS SINGLE)

Sets the width of the line stroke.

Parameters

sw - width

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF Stroke Width Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage

FOR y# = 1 TO 10 STEP .5
  pdfSetStrokeWidth y#
  pdfBeginPath 150, 550 - y# * 30
  pdfAppendLine 250, 550 - y# * 30
  pdfStrokePath
  pdfBeginText
  pdfSetFontSize "F1", 12
  pdfSetPosition 50, 547 - y# * 30
  pdfAddText "Stroke width : " + STR$(y#)
  pdfEndText
NEXT
pdfGen "pdfStrokeWidthExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetCharacterSpacing (s AS SINGLE)

Set the character spacing, which is a number expressed in unscaled text space units.

Parameters

s - unscaled text space units

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF Character Spacing Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition 150, 350 'Initial position
pdfSetLeading 15
pdfAddText "This is some text."
pdfNextLine
pdfSetCharacterSpacing 1
pdfAddText "This is some text."
pdfNextLine
pdfSetCharacterSpacing 2
pdfAddText "This is some text."
pdfNextLine
pdfSetCharacterSpacing 3
pdfAddText "This is some text."
pdfEndText

pdfGen "pdfCharacterSpacingExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetWordSpacing (s AS SINGLE)

Set the word spacing, to wordSpace, which is a number expressed in unscaled text space units.

Parameters

s - unscaled text space units

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF Word Spacing Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition 150, 650
pdfSetLeading 15
pdfAddText "This is some text."
pdfNextLine
pdfSetWordSpacing 1
pdfAddText "This is some text."
pdfNextLine
pdfSetWordSpacing 2
pdfAddText "This is some text."
pdfNextLine
pdfSetWordSpacing 3
pdfAddText "This is some text."
pdfNextLine
pdfSetWordSpacing 4
pdfAddText "This is some text."
pdfNextLine
pdfSetWordSpacing 5
pdfAddText "This is some text."
pdfEndText

pdfGen "pdfWordSpacingExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetHorizontalScaling (s AS LONG)

Set the horizontal scaling. Scale is a number specifying the percentage of the normal width. Initial value: 100 (normal width).

Parameters

s - percentage.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF Horizontal Scaling Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 48
pdfSetPosition 100, 720
pdfSetLeading 48
pdfSetHorizontalScaling 80
pdfAddText "This is some text."
pdfNextLine
pdfSetHorizontalScaling 90
pdfAddText "This is some text."
pdfNextLine
pdfSetHorizontalScaling 100
pdfAddText "This is some text."
pdfNextLine
pdfSetHorizontalScaling 110
pdfAddText "This is some text."
pdfNextLine
pdfSetHorizontalScaling 120
pdfAddText "This is some text."
pdfEndText

pdfGen "pdfHorizontalScalingExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetLeading (l AS SINGLE)

Sets the text leading in unscaled text space units. It specifies the vertical distance between the baselines of adjacent lines of text.

Parameters

l - Distance between lines.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF SetLeading Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetPosition 50, 720 'Initial position
pdfSetFontSize "F1", 14
pdfSetLeading 14
pdfAddText "This is some text."
pdfSetLeading 16
pdfNextLine
pdfAddText "This is some text."
pdfSetLeading 18
pdfNextLine
pdfAddText "This is some text."
pdfSetLeading 20
pdfNextLine
pdfAddText "This is some text."
pdfSetLeading 22
pdfNextLine
pdfAddText "This is some text."
pdfSetLeading 24
pdfNextLine
pdfAddText "This is some text."
pdfEndText

pdfGen "pdfSetLeadingExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetRise (r AS SINGLE)

Text rise specifies the distance, in unscaled text space units, to move the baseline up or down from its default location.

Parameters

r - Amount of rise (+/-)

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

  pdfSetTitle "PDF SetRise Example"
  pdfSetAuthor "Justsomeguy"

  pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

  pdfAddPage
  pdfBeginText
  pdfSetPosition 50, 720 'Initial position
  pdfSetFontSize "F1", 14
  pdfAddText "This is some text. "
  pdfSetFontSize "F1", 7
  pdfSetRise 7
  pdfAddText "This is superscript"
  pdfSetRise -3
  pdfAddText "This is subscript"
  pdfSetRise 0
  pdfSetFontSize "F1", 14
  pdfAddText "This is normal text"

  pdfEndText

  pdfGen "pdfSetRiseExample.pdf"
  SYSTEM
  '$INCLUDE:'pdfGen.bm'
```

SUB pdfRotateText (x AS LONG, y AS LONG, deg AS SINGLE)

Helper function to set the matrix for a rotation around a fixed cooridinate.

Parameters

x - position
y - position
deg - rotation in degrees

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF RotateText Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 14
FOR deg! = 0 TO 360 STEP 15
  pdfRotateText 300, 400, deg!
  pdfAddText "           This is some text. "
NEXT
pdfEndText

pdfGen "pdfRotateTextExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetMatrix (a AS SINGLE, b AS SINGLE, _
c AS SINGLE, d AS SINGLE, _
e AS SINGLE, f AS SINGLE)

Set the text matrix.

$$\begin{bmatrix} a & b & 0 \\ c & d & 0 \\ e & f & 1 \end{bmatrix} = [a\ b\ c\ d\ e\ f]$$

* Translations are specified as [1 0 0 1 tx ty], where tx and ty are the distances to translate the origin of the coordinate system in the horizontal and vertical dimensions, respectively.
* Scaling is obtained by [sx 0 0 sy 0 0]. This scales the coordinates so that 1 unit in the horizontal and vertical dimensions of the new coordinate system is the same size as sx and sy units, respectively, in the previous coordinate system.
* Rotations are produced by [cos ù sin ù -sin ù cos ù 0 0], which has the effect of rotating the coordinate system axes by an angle counterclockwise.
* Skew is specified by [1 tan α tan β 1 0 0], which skews the x axis by an angle α and the y axis by an angle β.

Parameters

a-f elements of the matrix.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF SetMatrix Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 36

xp! = 50 ' x position
yp! = 700 ' y position
pdfSetMatrix 1, 0, 0, 1, xp!, yp!
pdfAddText "This is normal text."

scaleX! = 2 ' x scale
scaleY! = 2 ' y scale
pdfSetMatrix scaleX!, 0, 0, scaleY!, xp!, yp! - 100.0
pdfAddText "This is scaled text. "

alpha! = 0.0 ' angle from x axis
beta! = 45.0 ' angle from y axis
pdfSetMatrix 1, TAN(_D2R(alpha!)), TAN(_D2R(beta!)), 1, xp!, yp! - 200
pdfAddText "This is skewed text. "

theta! = 30.0 ' angle of rotation
pdfSetMatrix COS(_D2R(theta!)), SIN(_D2R(theta!)),
-SIN(_D2R(theta!)), COS(_D2R(theta!)), xp!, yp! - 400
pdfAddText "This is rotated text. "

pdfEndText

pdfGen "pdfSetMatrixExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfRotateImage (x AS LONG, y AS LONG, deg AS SINGLE)

Helper function to set the matrix for a rotation around a fixed cooridinate.

Parameters

x - position
y - position
deg - angle in degrees around fixed point

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF Rotate Image Example"
pdfSetAuthor "Justsomeguy"

pdfLoadImage "Icon.png", "Im1"

pdfAddPage
FOR theta! = 0 TO 360 STEP 15
  pdfPushGraphicsStack
  pdfSetImageMatrix 100, 0, 0, 100, 300, 600
  pdfRotateImage 0, 0, theta!
  pdfPutImage "Im1"
  pdfPopGraphicsStack
NEXT

pdfGen "pdfRotateImage.pdf"

SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetImageMatrix (a AS SINGLE, b AS SINGLE, _
c AS SINGLE, d AS SINGLE, _
e AS SINGLE, f AS SINGLE)
Set the Image matrix.

$$\begin{bmatrix} a & b & 0 \\ c & d & 0 \\ e & f & 1 \end{bmatrix} = [a\ b\ c\ d\ e\ f]$$

* Translations are specified as [1 0 0 1 tx ty], where tx and ty are the distances to translate the origin of the coordinate system in the horizontal and vertical dimensions, respectively.
* Scaling is obtained by [sx 0 0 sy 0 0]. This scales the coordinates so that 1 unit in the horizontal and vertical dimensions of the new coordinate system is the same size as sx and sy units, respectively, in the previous coordinate system.
* Rotations are produced by [cos ù sin ù -sin ù cos ù 0 0], which has the effect of rotating the coordinate system axes by an angle counterclockwise.
* Skew is specified by [1 tan α tan β 1 0 0], which skews the x axis by an angle α and the y axis by an angle β.

Parameters

a-f elements of the matrix.

Returns

Nothing.

Example

```
 '$INCLUDE:'pdfGen.bi'
 pdfSetTitle "PDF SetImageMatrix Example"
 pdfSetAuthor "Justsomeguy"
 pdfLoadImage "Icon.png", "Im1"

 pdfAddPage
 'Normal Image
 pdfPushGraphicsStack
 pdfSetImageMatrix 100, 0, 0, 100, 250, 650
 pdfPutImage "Im1"
 pdfPopGraphicsStack
 'Scaled Image
 pdfPushGraphicsStack
 pdfSetImageMatrix 200, 0, 0, 200, 200, 400
 pdfPutImage "Im1"
 pdfPopGraphicsStack
 'Rotated Image
 deg! = 45
 pdfPushGraphicsStack
 pdfSetImageMatrix 100, 0, 0, 100, 300, 225
 pdfSetImageMatrix COS(_D2R(deg!)), SIN(_D2R(deg!)),
-SIN(_D2R(deg!)), COS(_D2R(deg!)), 0, 0
 pdfPutImage "Im1"
 pdfPopGraphicsStack
 'Skewed Image
 alpha! = 10
 beta! = -30
 pdfPushGraphicsStack
 pdfSetImageMatrix 100, 0, 0, 100, 300, 75
 pdfSetImageMatrix 1, TAN(_D2R(alpha!)), TAN(_D2R(beta!)), 1, 0, 0
 pdfPutImage "Im1"
 pdfPopGraphicsStack

 pdfGen "pdfImageMatrix.pdf"
  '_pdf_T SYSTEM
 '$INCLUDE:'pdfGen.bm'
```

SUB pdfPutImage (nm AS STRING)

Places an image on the current page.

Parameters

nm - String with the ID of the image.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF PutImage Example"
pdfSetAuthor "Justsomeguy"
pdfLoadImage "Icon.png", "Im1"

pdfAddPage

scaleX! = 100
scaleY! = 100
xP! = 250
yP! = 650

pdfPushGraphicsStack
pdfSetImageMatrix scaleX!, 0, 0, scaleY!, xP!, yP!
pdfPutImage "Im1"
pdfPopGraphicsStack

pdfGen "pdfPutImage.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetFontSize (nm AS STRING, sz AS LONG)

Sets the current loaded font and size.

Parameters

nm - ID of font that was previously loaded.
sz - size of font

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF SetFontSize Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage

pdfBeginText
pdfSetPosition 50, 750 'Initial position
FOR i& = 0 TO 28
  fSize& = 8 + i&
  pdfSetFontSize "F1", fSize&
  pdfSetLeading fSize&
  pdfAddText "This is fontSize " + STR$(fSize&) + "."
  pdfNextLine
NEXT
pdfEndText

pdfGen "pdfSetFontSizeExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfBeginPath (x AS LONG, y AS LONG)

A path is a set sequential points in which a draw command renders them to the current page. All paths must have a starting point. pdfBeginPath sets the starting point of a path.

Parameters

x,y - position of the start of the path

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF BeginPath Example"
pdfSetAuthor "Justsomeguy"

pdfAddPage
xC! = 300
yC! = 350
rad! = 250
deg! = 0

pdfBeginPath xC! + rad! * SIN(_D2R(deg!)), yC! + rad! * COS(_D2R(deg!))

FOR i& = 0 TO 249
  deg! = deg! + 91
  rad! = rad! - 1
  pdfAppendLine xC! + rad! * SIN(_D2R(deg!)), yC! + rad! * COS(_D2R(deg!))
NEXT
pdfStrokePath

pdfGen "pdfBeginPathExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfAppendLine (x1 AS LONG, y1 AS LONG)

Adds a straight line to previously started path.

Parameters

x1,y1 - Position of the end point of a line

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF AppendLine Example"
pdfSetAuthor "Justsomeguy"

pdfAddPage
xC! = 300
yC! = 350
rad! = 250
deg! = 0

pdfBeginPath xC! + rad! * SIN(_D2R(deg!)), yC! + rad! * COS(_D2R(deg!))
FOR i& = 0 TO 249
  deg! = deg! + 91
  rad! = rad! - 1
  pdfAppendLine xC! + rad! * SIN(_D2R(deg!)), yC! + rad! * COS(_D2R(deg!))
NEXT
pdfStrokePath

pdfGen "pdfAppendLineExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfAppendBezier123 (x1 AS LONG, y1 AS LONG, _
x2 AS LONG, y2 AS LONG, _
x3 AS LONG, y3 AS LONG)

Append a cubic Bezier curve to the current path. The curve extends from the current point to the point (x3 , y3), using (x1 , y1) and (x2 , y2) as the Bezier control points. The new current point is (x3 , y3)

Parameters

x1,y1 - Control point
x2,y2 - Control point
x3,y3 - Final point

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF AppendBezier123 Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
'Starting Point
xC! = 300
yC! = 350
'Control Point 1
cPx1! = xC! - 50
cPy1! = yC! + 200
'Control Point 2
cPx2! = xC!
cPy2! = yC! + 250
'Final Point
fNx! = xC! + 200
fNy! = yC! + 200
'draw Bezier
pdfBeginPath xC!, yC!
pdfAppendBezier123 cPx1!, cPy1!, cPx2!, cPy2!, fNx!, fNy!
pdfStrokePath
'draw visual aids
pdfRectangle xC! - 2, yC! - 2, 4, 4
pdfStrokePath
pdfRectangle fNx! - 2, fNy! - 2, 4, 4
pdfStrokePath
pdfRectangle cPx1! - 2, cPy1! - 2, 4, 4
pdfStrokePath
pdfRectangle cPx2! - 2, cPy2! - 2, 4, 4
pdfStrokePath
pdfBeginPath cPx1!, cPy1!
pdfSetLineDashPattern 2, 2, 0
pdfAppendLine xC!, yC!
pdfStrokePath
pdfBeginPath cPx2!, cPy2!
pdfSetLineDashPattern 2, 2, 0
pdfAppendLine fNx!, fNy!
pdfStrokePath
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition xC! - 40, yC - 15!
pdfAddText "Starting Point"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition cPx1! - 40, cPy1! + 5
pdfAddText "Control Point 1"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition cPx2! - 40, cPy2! + 5
pdfAddText "Control Point 2"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition fNx! - 30, fNy! - 15
pdfAddText "End Point"
pdfEndText
pdfGen "pdfAppendBezier123Example.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfAppendBezier23 (x2 AS LONG, y2 AS LONG, _
x3 AS LONG, y3 AS LONG)

Append a cubic Bezier curve to the current path. The curve extends from the current point to the point (x3, y3), using the point (x2, y2) as the Bezier control points. The new current point is (x3, y3).

Parameters

x2,y2 - control point
x3,y3 - end point

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF AppendBezier23 Example"
pdfSetAuthor "Justsomeguy"
pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
'Starting Point
xC! = 300
yC! = 350
'Control Point 1
cPx1! = xC! + 50
cPy1! = yC! + 250
'Final Point
fNx! = xC! + 200
fNy! = yC! + 200
'draw Bezier
pdfBeginPath xC!, yC!
pdfAppendBezier23 cPx1!, cPy1!, fNx!, fNy!
pdfStrokePath
'draw visual aids
pdfRectangle xC! - 2, yC! - 2, 4, 4
pdfStrokePath
pdfRectangle fNx! - 2, fNy! - 2, 4, 4
pdfStrokePath
pdfRectangle cPx1! - 2, cPy1! - 2, 4, 4
pdfStrokePath
pdfBeginPath cPx1!, cPy1!
pdfSetLineDashPattern 2, 2, 0
pdfAppendLine fNx!, fNy!
pdfStrokePath
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition xC! - 40, yC - 15!
pdfAddText "Starting Point"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition cPx1! - 40, cPy1! + 5
pdfAddText "Control Point"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition fNx! - 30, fNy! - 15
pdfAddText "End Point"
pdfEndText

pdfGen "pdfAppendBezier23Example.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfAppendBezier13 (x1 AS LONG, y1 AS LONG, _
x3 AS LONG, y3 AS LONG)

Append a cubic Bezier curve to the current path. The curve extends from the current point to the point (x3, y3), using (x1, y1) and (x3, y3) as the Bezier control points. The new current point is (x3, y3).

Parameters

x1,y1 - Control point
x3,y3 - End Point

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF AppendBezier13 Example"
pdfSetAuthor "Justsomeguy"
pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
'Starting Point
xC! = 300
yC! = 350
'Control Point 1
cPx1! = xC! - 50
cPy1! = yC! + 150
'Final Point
fNx! = xC! + 200
fNy! = yC! + 200
'draw Bezier
pdfBeginPath xC!, yC!
pdfAppendBezier13 cPx1!, cPy1!, fNx!, fNy!
pdfStrokePath
'draw visual aids
pdfRectangle xC! - 2, yC! - 2, 4, 4
pdfStrokePath
pdfRectangle fNx! - 2, fNy! - 2, 4, 4
pdfStrokePath
pdfRectangle cPx1! - 2, cPy1! - 2, 4, 4
pdfStrokePath
pdfBeginPath cPx1!, cPy1!
pdfSetLineDashPattern 2, 2, 0
pdfAppendLine xC!, yC!
pdfStrokePath
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition xC! - 40, yC - 15!
pdfAddText "Starting Point"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition cPx1! - 40, cPy1! + 5
pdfAddText "Control Point"
pdfEndText
pdfBeginText
pdfSetFontSize "F1", 14
pdfSetPosition fNx! - 30, fNy! - 15
pdfAddText "End Point"
pdfEndText

pdfGen "pdfAppendBezier13Example.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfClosePath

Close the current subpath by appending a straight line segment from the current point to the starting point of the subpath.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF ClosePath Example"
pdfSetAuthor "Justsomeguy"

pdfAddPage
xC! = 300
yC! = 350
pdfBeginPath xC! + 5, yC! 'Start the path
pdfAppendLine xC! + 100, yC! + 100 'Points along the path
pdfAppendLine xC! + 100, yC! - 100 'Points along the path
pdfClosePath 'close path to the start

pdfBeginPath xC! - 5, yC! 'Start the path
pdfAppendLine xC! - 100, yC! - 100 'Points along the path
pdfAppendLine xC! - 100, yC! + 100 'Points along the path
pdfClosePath 'close path to the start

pdfStrokePath ' Path is not drawn until now

pdfGen "pdfClosePathExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfRectangle (x AS LONG, y AS LONG, _
w AS LONG, h AS LONG)

Append a rectangle to the current path as a complete subpath, with lower-left corner (x, y) and dimensions width and height in user space.

Parameters

x,y - lower-left corner
w,h - width and height

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF Rectangle Example"
pdfSetAuthor "Justsomeguy"

pdfAddPage
xC! = 150
yC! = 350
w! = 300
h! = 300

pdfSetStrokeWidth 4
FOR i& = 0 TO 100 STEP 10
  pdfRectangle xC! + i&, yC! + i&, w! - (i& * 2), h! - (i& * 2)
  pdfStrokePath
NEXT

pdfGen "pdfRectangleExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
```

SUB pdfStrokePath

Stroke the previously created path. It does not close the path. Subsequent path operations will continue from the last path operation.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF General Path Showcase Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfSetStrokeWidth 4

pdfBeginText
pdfSetFontSize "F1", 18
pdfSetPosition 100, 710
pdfAddText "StrokePath Example (Does not close shape)"
pdfEndText

pdfBeginPath 100, 700
pdfAppendLine 500, 700
pdfAppendLine 500, 650
pdfAppendLine 100, 650
pdfStrokePath

pdfBeginText
pdfSetPosition 100, 610
pdfAddText "CloseStrokePath Example (Closes shape)"
pdfEndText

pdfBeginPath 100, 600
pdfAppendLine 500, 600
pdfAppendLine 500, 550
pdfAppendLine 100, 550
pdfCloseStrokePath

pdfBeginText
pdfSetPosition 100, 510
pdfAddText "FillPath Example (Fills shape)"
pdfEndText

pdfSetStrokeWidth 1
rad! = 75: xc! = 300: yc! = 420
star rad!, xc!, yc!
pdfFillPath

pdfBeginText
pdfSetPosition 100, 300
pdfAddText "FillEO Example (Fills Even Odd Shape)"
pdfEndText

rad! = 75: xc! = 300: yc! = 200
star rad!, xc!, yc!
pdfFillEO

pdfGen "pdfPathShowcaseExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
SUB star (rad!, xc!, yc!)
  pdfBeginPath xc! + rad! * SIN(0), yc! + rad! * COS(0)
  FOR i! = 1 TO 5
  pdfAppendLine xc! + rad! * SIN(_D2R(i! * 144)), yc! + rad! * COS(_D2R(i! * 144))
  NEXT
END SUB
```

SUB pdfCloseStrokePath

Stroke the previously created path and closes the path.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF General Path Showcase Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfSetStrokeWidth 4

pdfBeginText
pdfSetFontSize "F1", 18
pdfSetPosition 100, 710
pdfAddText "StrokePath Example (Does not close shape)"
pdfEndText

pdfBeginPath 100, 700
pdfAppendLine 500, 700
pdfAppendLine 500, 650
pdfAppendLine 100, 650
pdfStrokePath

pdfBeginText
pdfSetPosition 100, 610
pdfAddText "CloseStrokePath Example (Closes shape)"
pdfEndText

pdfBeginPath 100, 600
pdfAppendLine 500, 600
pdfAppendLine 500, 550
pdfAppendLine 100, 550
pdfCloseStrokePath

pdfBeginText
pdfSetPosition 100, 510
pdfAddText "FillPath Example (Fills shape)"
pdfEndText

pdfSetStrokeWidth 1
rad! = 75: xc! = 300: yc! = 420
star rad!, xc!, yc!
pdfFillPath

pdfBeginText
pdfSetPosition 100, 300
pdfAddText "FillEO Example (Fills Even Odd Shape)"
pdfEndText

rad! = 75: xc! = 300: yc! = 200
star rad!, xc!, yc!
pdfFillEO

pdfGen "pdfPathShowcaseExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
SUB star (rad!, xc!, yc!)
  pdfBeginPath xc! + rad! * SIN(0), yc! + rad! * COS(0)
  FOR i! = 1 TO 5
  pdfAppendLine xc! + rad! * SIN(_D2R(i! * 144)), yc! + rad! * COS(_D2R(i! * 144))
  NEXT
END SUB
```

SUB pdfFillPath

Fill the path, using the nonzero winding number rule to determine the region to fill. Any subpaths that are open are implicitly closed before being filled.

Nonzero Winding Number Rule :

The nonzero winding number rule determines whether a given point is inside a path by conceptually drawing a ray from that point to infinity in any direction and then examining the places where a segment of the path crosses the ray. Starting with a count of 0, the rule adds 1 each time a path segment crosses the ray from left to right and subtracts 1 each time a segment crosses from right to left. After counting all the crossings, if the result is 0, the point is outside the path; otherwise, it is inside.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF General Path Showcase Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfSetStrokeWidth 4

pdfBeginText
pdfSetFontSize "F1", 18
pdfSetPosition 100, 710
pdfAddText "StrokePath Example (Does not close shape)"
pdfEndText

pdfBeginPath 100, 700
pdfAppendLine 500, 700
pdfAppendLine 500, 650
pdfAppendLine 100, 650
pdfStrokePath

pdfBeginText
pdfSetPosition 100, 610
pdfAddText "CloseStrokePath Example (Closes shape)"
pdfEndText

pdfBeginPath 100, 600
pdfAppendLine 500, 600
pdfAppendLine 500, 550
pdfAppendLine 100, 550
pdfCloseStrokePath

pdfBeginText
pdfSetPosition 100, 510
pdfAddText "FillPath Example (Fills shape)"
pdfEndText

pdfSetStrokeWidth 1
rad! = 75: xc! = 300: yc! = 420
star rad!, xc!, yc!
pdfFillPath

pdfBeginText
pdfSetPosition 100, 300
pdfAddText "FillEO Example (Fills Even Odd Shape)"
pdfEndText

rad! = 75: xc! = 300: yc! = 200
star rad!, xc!, yc!
pdfFillEO

pdfGen "pdfPathShowcaseExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
SUB star (rad!, xc!, yc!)
  pdfBeginPath xc! + rad! * SIN(0), yc! + rad! * COS(0)
  FOR i! = 1 TO 5
  pdfAppendLine xc! + rad! * SIN(_D2R(i! * 144)), yc! + rad! * COS(_D2R(i! * 144))
  NEXT
END SUB
```

SUB pdfFillEO

Fill the path, using the even-odd rule to determine the region to fill.

Even-Odd Rule :

An alternative to the nonzero winding number rule is the even-odd rule. This rule determines whether a point is inside a path by drawing a ray from that point in any direction and simply counting the number of path segments that cross the ray, regardless of direction. If this number is odd, the point is inside; if even, the point is outside. This yields the same results as the nonzero winding number rule for paths with simple shapes, but produces different results for more complex shapes.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF General Path Showcase Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfSetStrokeWidth 4

pdfBeginText
pdfSetFontSize "F1", 18
pdfSetPosition 100, 710
pdfAddText "StrokePath Example (Does not close shape)"
pdfEndText

pdfBeginPath 100, 700
pdfAppendLine 500, 700
pdfAppendLine 500, 650
pdfAppendLine 100, 650
pdfStrokePath

pdfBeginText
pdfSetPosition 100, 610
pdfAddText "CloseStrokePath Example (Closes shape)"
pdfEndText

pdfBeginPath 100, 600
pdfAppendLine 500, 600
pdfAppendLine 500, 550
pdfAppendLine 100, 550
pdfCloseStrokePath

pdfBeginText
pdfSetPosition 100, 510
pdfAddText "FillPath Example (Fills shape)"
pdfEndText

pdfSetStrokeWidth 1
rad! = 75: xc! = 300: yc! = 420
star rad!, xc!, yc!
pdfFillPath

pdfBeginText
pdfSetPosition 100, 300
pdfAddText "FillEO Example (Fills Even Odd Shape)"
pdfEndText

rad! = 75: xc! = 300: yc! = 200
star rad!, xc!, yc!
pdfFillEO

pdfGen "pdfPathShowcaseExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
SUB star (rad!, xc!, yc!)
  pdfBeginPath xc! + rad! * SIN(0), yc! + rad! * COS(0)
  FOR i! = 1 TO 5
  pdfAppendLine xc! + rad! * SIN(_D2R(i! * 144)), yc! + rad! * COS(_D2R(i! * 144))
  NEXT
END SUB
```

SUB pdfFillStroke

Place Holder Text.

Parameters

None.

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'
pdfSetTitle "PDF General Path Showcase Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfAddPage
pdfSetStrokeWidth 4

pdfBeginText
pdfSetFontSize "F1", 18
pdfSetPosition 100, 710
pdfAddText "StrokePath Example (Does not close shape)"
pdfEndText

pdfBeginPath 100, 700
pdfAppendLine 500, 700
pdfAppendLine 500, 650
pdfAppendLine 100, 650
pdfStrokePath

pdfBeginText
pdfSetPosition 100, 610
pdfAddText "CloseStrokePath Example (Closes shape)"
pdfEndText

pdfBeginPath 100, 600
pdfAppendLine 500, 600
pdfAppendLine 500, 550
pdfAppendLine 100, 550
pdfCloseStrokePath

pdfBeginText
pdfSetPosition 100, 510
pdfAddText "FillPath Example (Fills shape)"
pdfEndText

pdfSetStrokeWidth 1
rad! = 75: xc! = 300: yc! = 420
star rad!, xc!, yc!
pdfFillPath

pdfBeginText
pdfSetPosition 100, 300
pdfAddText "FillEO Example (Fills Even Odd Shape)"
pdfEndText

rad! = 75: xc! = 300: yc! = 200
star rad!, xc!, yc!
pdfFillEO

pdfGen "pdfPathShowcaseExample.pdf"
SYSTEM
'$INCLUDE:'pdfGen.bm'
SUB star (rad!, xc!, yc!)
  pdfBeginPath xc! + rad! * SIN(0), yc! + rad! * COS(0)
  FOR i! = 1 TO 5
  pdfAppendLine xc! + rad! * SIN(_D2R(i! * 144)), yc! + rad! * COS(_D2R(i! * 144))
  NEXT
END SUB
```

SUB pdfFillStrokeEO

Fill the path, using the even-odd rule to determine the region to fill.

Parameters

None.

Returns

Nothing.

Example

SUB pdfCloseFillStroke

Close, fill, and then stroke the path, using the nonzero winding number rule to determine the region to fill.

Parameters

None.

Returns

Nothing.

Example

SUB pdfCloseFillStrokeEO

Close, fill, and then stroke the path, using the even-odd rule to determine the region to fill.

Parameters

None.

Returns

Nothing.

Example

SUB pdfEndPath

End the path object without filling or stroking it. This operator is a "path-painting no-op," used primarily for the side effect of changing the current clipping path.

Parameters

None.

Returns

Nothing.

Example

SUB pdfSetColorStroke (r AS SINGLE, _
g AS SINGLE, _
b AS SINGLE)

Set the RGB color of the stroke. Values are between 0.0 - 1.0

Parameters

r,g,b - 0.0 to 1.0 (0.0 is dark and 1.0 is bright)

Returns

Nothing.

Example

SUB pdfSetColorNonStroke (r AS SINGLE, _
g AS SINGLE, _
b AS SINGLE)

Set the RGB color of the non-stroke operations such as fill. Values are between 0.0 - 1.0

Parameters

None.

Returns

Nothing.

Example

SUB pdfPushGraphicsStack

A well-structured PDF document typically contains many graphical elements that are essentially independent of each other and sometimes nested to multiple levels. The graphics state stack allows these elements to make local changes to the graphics state without disturbing the graphics state of the surrounding environment. The stack is a LIFO (last in, first out) data structure in which the contents of the graphics state can be saved and later restored.

Parameters

None.

Returns

Nothing.

Example

SUB pdfPopGraphicsStack

A well-structured PDF document typically contains many graphical elements that are essentially independent of each other and sometimes nested to multiple levels. The graphics state stack allows these elements to make local changes to the graphics state without disturbing the graphics state of the surrounding environment. The stack is a LIFO (last in, first out) data structure in which the contents of the graphics state can be saved and later restored.

Parameters

None.

Returns

Nothing.

Example

SUB pdfMediaBox (x AS LONG, y AS LONG, _
x1 AS LONG, y1 AS LONG)

The media box defines the boundaries of the physical medium on which the page is to be printed. It may include any extended area surrounding the finished page for bleed, printing marks, or other such purposes. Must be done before page is added.

The default values are 0,0,612,792. Changing the media box will automatically set the crop box to the same values.

Parameters

x,y - lower left corner
x1,y1 - upper right corner

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF MediaBox Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN
pdfMediaBox 0, 0, 2048, 600

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 24
pdfSetPosition 900, 500
pdfAddText "Wide Page"
pdfEndText

pdfMediaBox 0, 0, 600, 2048

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 24
pdfSetPosition 270, 1948
pdfAddText "Long Page"
pdfEndText

pdfGen "pdfMediaBox.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfCropBox (x AS LONG, y AS LONG, _
x1 AS LONG, y1 AS LONG)

The crop box defines the region to which the contents of the page are to be clipped (cropped) when displayed or printed. Unlike the other boxes, the crop box has no defined meaning in terms of physical page geometry or intended use; it merely imposes clipping on the page contents. Must be done before page is added.
The default values are 0,0,612,792.

Parameters

x,y - lower left corner
x1,y1 - upper right corner

Returns

Nothing.

Example

```
'$INCLUDE:'pdfGen.bi'

pdfSetTitle "PDF CropBox Example"
pdfSetAuthor "Justsomeguy"

pdfLoadFont "F1", PDF_FONT_TIMES_ROMAN

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 100
pdfSetPosition 25, 350
pdfAddText "Cropped Text"
pdfEndText

pdfCropBox 100, 100, 512, 692

pdfAddPage
pdfBeginText
pdfSetFontSize "F1", 100
pdfSetPosition 25, 350
pdfAddText "Cropped Text"
pdfEndText

pdfGen "pdfCropBox.pdf"
SYSTEM

'$INCLUDE:'pdfGen.bm'
```

SUB pdfSetImageDevice (imdev AS _UNSIGNED _BYTE)

Sets the current image device. Specifying colors in a device color space (DeviceGray, DeviceRGB, or DeviceCMYK) makes them device-dependent. By setting default color spaces (PDF 1.1), a PDF document can request that such colors be systematically transformed (remapped) into device-independent CIE-based color spaces.

Parameters

imdev - PDF_IMAGE_DEVICE_GRAY, PDF_IMAGE_DEVICE_RGB, or PDF_IMAGE_DEVICE_CMYK are valid contants.

Returns

Nothing.

Example

SUB pdfLoadImage (f AS STRING, nm AS STRING)

Loads an image and prepares it for use in the document.

Parameters

f - Filename of image
nm - ID of image to be used later

Returns

Nothing.

Example

SUB pdfSetTitle (s AS STRING)

Sets the title of the pdf. Depending on the viewer It can be seen in the properties of the pdf.

Parameters

s - Title

Returns

Nothing.

Example

SUB pdfSetSubject (s AS STRING)

Sets the subject of the pdf. Depending on the viewer It can be seen in the properties of the pdf.

Parameters

s - Subject

Returns

Nothing.

Example

SUB pdfSetAuthor (s AS STRING)

Sets the author of the pdf. Depending on the viewer It can be seen in the properties of the pdf.

Parameters

s - Author.

Returns

Nothing.

Example

SUB pdfSetKeywords (s AS STRING)

Sets the keywords of the pdf. Depending on the viewer It can be seen in the properties of the pdf.

Parameters

s - Keywords.

Returns

Nothing.

Example

SUB pdfSetCreator (s AS STRING)

Sets the creator of the pdf. Depending on the viewer It can be seen in the properties of the pdf.

Parameters

s - Creator.

Returns

Nothing.

Example

SUB pdfSetProducer (s AS STRING)

Sets the producer of the pdf. Depending on the viewer It can be seen in the properties of the pdf.

Parameters

s - producer.

Returns

Nothing.

Example

SUB pdfSetLineCapStyle (c AS LONG)

The line cap style specifies the shape to be used at the ends of open subpaths (and dashes, if any) when they are stroked.

Parameters

c - PDF_LINE_CAP_BUTT, PDF_LINE_CAP_ROUND, or PDF_LINE_CAP_PROJECTING_SQUARE

Returns

Nothing.

Example

SUB pdfSetLineJoinStyle (c AS LONG)

The line join style specifies the shape to be used at the corners of paths that are stroked.

Parameters

c - PDF_LINE_JOIN_MITER, PDF_LINE_JOIN_ROUND, or PDF_LINE_JOIN_BEVEL

Returns

Nothing.

Example

SUB pdfSetLineDashPattern (ondash AS LONG, offdash AS LONG, phase AS LONG)

The line dash pattern controls the pattern of dashes and gaps used to stroke paths. It is specified by a dash array and a dash phase. The dash array's elements are numbers that specify the lengths of alternating dashes and gaps; the dash phase specifies the distance into the dash pattern at which to start the dash. The elements of both the dash array and the dash phase are expressed in user space units.

Parameters

ondash - number of units ON
offdash - number of units OFF
phase - units of offset

Returns

Nothing.

Example

SUB pdfGen (filename AS STRING)

Generates the PDF document.

Parameters

Filename - string

Returns

PDF file.

Example
